

Spring 2026 ME/CS/ECE759 Final Project Report
University of Wisconsin-Madison

Parallelising an Obstacle Avoidance Algorithm using Histogram of Gradient and Support Vector Machine

Ashvin Anilkumar

May 5, 2026

Abstract

Traditional obstacle avoidance algorithms use neural networks for detecting objects, which requires more hardware utilization. In this project, we explore a path that is light on the hardware - a combination of two techniques - the Histogram of Gradients (HOG) and Support Vector Machine (SVM). This project explores how good a speed boost we will get when we try to parallelise an obstacle avoidance algorithm using HOG and SVM. We compare the inference time and accuracy in detecting the rocks for a Mars Rover, helping in navigating the vehicle in a debris-free way.

Link to Final Project git repo: [Link](#)

Contents

1. General information	4
2. Problem statement	4
3. Solution description	4
4. Overview of results. Demonstration of your project	6
5. Deliverables. Building & running your project	8
6. Conclusions and Future Work	9
References	10

1. General information

In this short section, please provide only the following information, in bulleted form (six bullets) and in this order:

1. Name: Ashvin Anilkumar
2. Email: anilkumar4@wisc.edu
3. Home department: ECE
4. Status: MS Student
5. I release the ME759 Final Project code as open source and under a BSD3 license for unfettered use by any interested party.

2. Problem statement

The objective of this project is to design, implement, and benchmark a C++ based Histogram of Oriented Gradients (HOG) + SVM object detection pipeline, successfully transitioning it from a sequential execution model to a parallelized architecture. The primary goal is to drastically reduce per-frame inference latency and increase Frames Per Second (FPS) without compromising detection accuracy.

In the last semester, I worked on developing an obstacle avoidance and object detection algorithm for the Mars rover developed by Wisconsin Robotics. In there, we used a depth camera, Luxonis OAK-D, which had an API with which we could use the depth value received from the camera to calculate obstacles that are in front of the camera. I remembered that, and I thought, “What if there is a better and simpler way to implement what I did then?” That’s when I came across Histogram of Oriented Gradients (HOG)[1].

While deep learning dominates modern computer vision, classical algorithms like Histogram of Oriented Gradients (HOG) paired with a Support Vector Machine (SVM)[2] remain highly relevant for low-power edge devices where heavy neural networks are impractical. The primary bottleneck of HOG+SVM is its "sliding window" approach—sequentially computing gradients for thousands of overlapping image patches. Because calculating the features of one window does not depend on the others, this is an "embarrassingly parallel" problem. Optimizing this pipeline offers a highly measurable, real-world application of parallel computing.

3. Solution description

I started by searching for a dataset that is suitable for training this obstacle avoidance algorithm, which I’m training to detect rocks and avoid them. Since it is very hard to distinguish between rocks and debris in Martian terrain, the data must be of the highest quality. Finding a dataset for this specific use case turned out to be really challenging - the existing datasets only had images of rocks that were close-ups or those that had just a single type of rock. Since I wanted this system to be able to detect any kind of rock - irrespective of the materials with which it is made - I decided to create my own dataset.

I searched the internet for rocks in Martian terrain, but the quality of the images was poor. Then I decided to take a new approach - generation of a synthetic dataset using diffusion models such as GPT 4o and Nano Banana - and the results were really good. The dataset is attached in the deliverables section of this document. Since building a custom dataset was a bit cumbersome, I decided not to use video for benchmarking purposes, since it would be challenging - I will attempt it only if I have the bandwidth to do so.

I began working on building the project by first defining my specs clearly. Some of the parameters that I fixed based on my application are shown below. I used generative AI for finalising the values of these parameters:

Parameter	Value
Window Size	64x64 px
Cell Size	8x8 px
Block Size	16x16 px (Each block has 2x2 cells)
Block Stride	8 px
Orientation bins	9 (unsigned, 0-180°)
Blocks per window	49 (7x7)
Feature vector Dimension	1764 (49 x 4 cells x 9 bins)

After finalising the specs, I prepared scripts for preprocessing the image dataset that was used for training the SVM. All the images in the dataset were converted into 64x64 crop images and split into training and testing sets. In each set, it had positive crops and negative crops. A positive crop contains the portion of the image that is inside the bounding box with some padding for context, whereas a negative crop contains random images that are not rocks.

For training the SVM, I implemented a training pipeline that generates a weights.bin and bias.txt, which are the metadata of the trained SVM. It first loads all the crops from the training and runs Histogram of Gradient (HOG) in it. This generates a 1764-dimensional feature vector, which is then passed on to sklearn's LinearSVM for training the SVM.

The main bottleneck of this process is the sliding window, which, just like convolution, could be parallelised. When it comes to the sliding window in this scenario, which is 64x64 px, it calls the HOG, runs through the trained SVM model, and classifies as a rock or not based on the confidence score. I've implemented the OpenMP version of the sliding window as well by flattening the 2D sliding window grid into a 1D thread pool using **collapse**. I've also implemented the CUDA version of the sliding window, but it was not quite straightforward. I had to do the preprocessing split on the host side because of the nvcc version that I was using on my local machine – nvcc 11.5 paired with GCC 11 cannot parse

OpenCV's headers under C++17 — OpenCV transitively includes GCC 11's standard library headers, which use `std::function` in ways `nvcc` cannot handle. So all OpenCV code lives in a plain `.cpp` file compiled by GCC, while the `.cu` file sees only raw pointers and CUDA headers.

For the CUDA version of the algorithm, the host converts the input frame to a float32 grayscale buffer and copies it to the GPU, then launches a 3D HOG kernel with grid dimensions `(n_win_x, n_win_y, 49)` — one CUDA block per (window position, HOG block) pair — where each block's 256 threads compute gradients, fractionally vote magnitudes into 8 warp-private shared-memory histograms to minimize **atomicAdd** contention, reduce those 8 histograms into a single 36-bin block histogram, apply L2-Hys normalization, and write the result into a global feature array of shape `[n_wins x 1764]`; a second SVM kernel then launches with grid `(n_win_x, n_win_y)`, where each block's 256 threads compute a strided partial dot product over the 1764-dim feature vector, tree-reduce it in shared memory, add the bias, and — if the score exceeds the threshold — atomically claim a slot in the output detection array, which is finally copied back to the host.

In the quest for improving the F1 score of SVM, which was unexpectedly hard, I've also implemented some helper functions, such as `f1_diagnosis`, which helped me get an insight into how I can get the F1 score above 0.85. The repository now has an SVM with an F1 score of 0.8544.

After generating `weights.bin` and `bias.txt` files, which are basically the key components of the SVM, we use them with the HOG extraction algorithm to detect the rocks in the image — HOG processes a 64x64 float32 grayscale patch and yields a 1764-dimensional descriptor vector. We pass that vector to the SVM, which will classify whether or not that vector represents a rock. After detection, greedy IoU-based Non-Maximum Suppression (NMS) runs on the CPU. Candidates are sorted by score in a descending manner; each box not yet suppressed is kept and used to suppress all subsequent boxes with `IoU > 0.4`. Because the SVM threshold filters out the vast majority of windows, NMS operates on a small set of candidates and adds negligible latency.

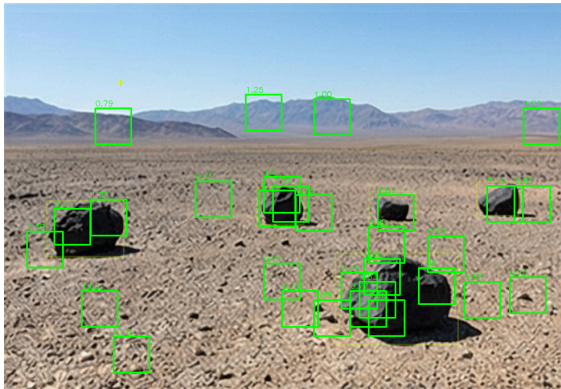
4. Overview of results. Demonstration of your project

With the help of generative AI, I completed the objectives that were promised in the proposal. The implementation can take in an input image, run inference using any of the 3 modes — CUDA, OpenMP, or sequential — and give out the result, where it will detect the rocks in the image. Since the SVM was trained correctly, we get great results when testing out the accuracy of the predictions made. One thing I've noticed is that sometimes the SVM classifies faraway mountains and dirt in the ground as rocks. The detection algorithm is not perfect, but it could be made even better by improving the F1 score of the SVM.

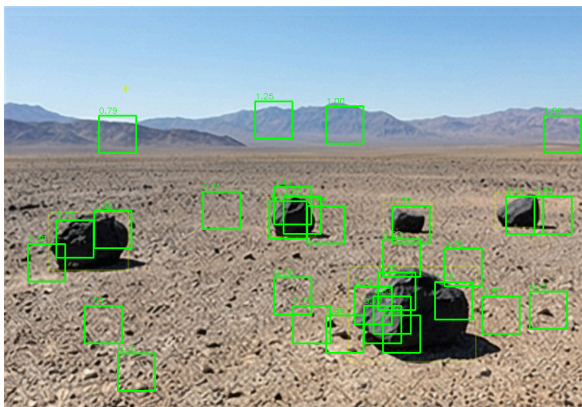
Shown below are some of the outputs I got while conducting some test runs in the project. The output is almost identical irrespective of the mode of inference, but the latency is drastically different.



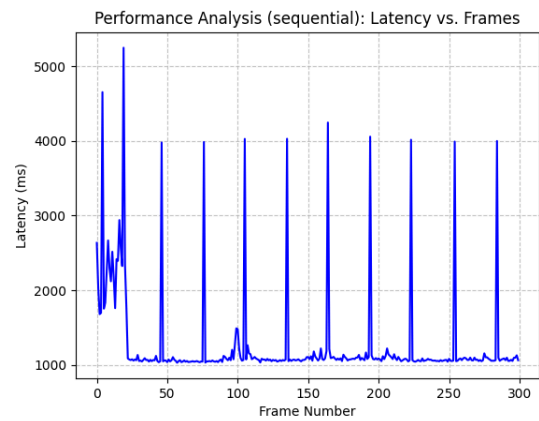
Input Image



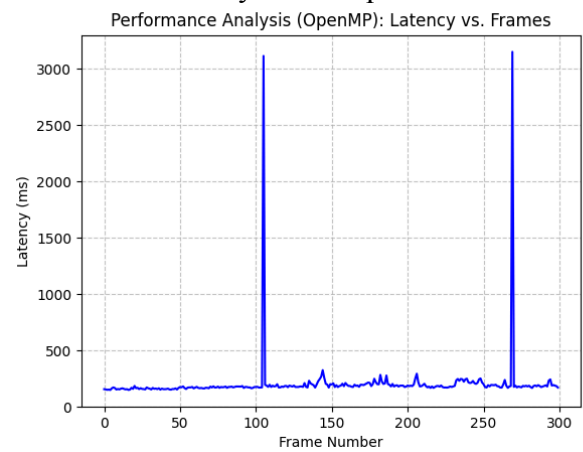
Output image from sequential execution



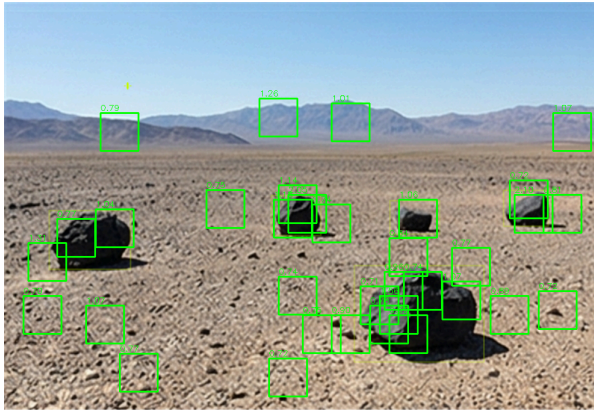
Output image from OpenMP execution



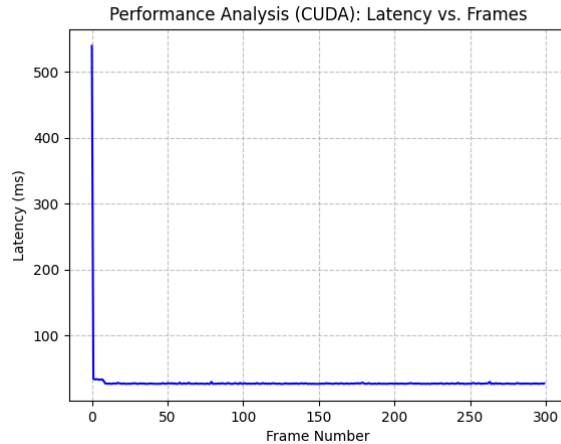
Performance Analysis of sequential execution



Performance Analysis of OpenMP execution



Output image from CUDA execution



Performance Analysis of CUDA execution

5. Deliverables. Building & running your project

Synthetic Rock Dataset I created – [Link](#)

Github Repository – [Link](#)

Initial setup:

1. git clone git@github.com:ashvin-a/FinalProject.git
2. cd FinalProject/
3. mkdir build && cd build
4. cmake ..
5. make -j\$(nproc)
6. cd ../
7. python3 -m venv env && source env/bin/activate
8. pip install -r requirements.txt

Try out the Rock detection using CUDA/OpenMP/Sequential

1. ./build/hog_detector input_image.png project/models/weights.bin project/models/bias.txt
project/results/output_cuda.png --mode cuda # For running in CUDA
2. ./build/hog_detector input_image.png project/models/weights.bin project/models/bias.txt
project/results/output_cuda.png --mode omp # For running in OpenMP
3. ./build/hog_detector input_image.png project/models/weights.bin project/models/bias.txt
project/results/output_cuda.png --mode seq # For running in Sequential mode.

To enable profiling:

1. cd build
2. cmake .. -DENABLE_PROFILING=ON
3. make -j\$(nproc)
4. cd ../
5. ./build/hog_detector input_image.png project/models/weights.bin project/models/bias.txt
project/results/output_cuda.png --mode cuda
6. gprof ./build/hog_detector gmon.out > profile.txt

For downloading and preparing the dataset

1. `cd project`
2. `mkdir data && cd data`
3. `curl -L "https://app.roboflow.com/ds/AVwkFhBSLM?key=CvWn4MvvWE" > roboflow.zip; unzip roboflow.zip; rm roboflow.zip`
4. `cd ../../ # Should come back to main directory`
5. `python project/scripts/prepare_dataset.py --data_dir project/data/roboflow --out_dir project/data/crops`

Training the SVM

1. `python project/scripts/train_svm.py --crops_dir project/data/crops --out_dir project/models`

Generate plots for the runs

1. `python project/scripts/plots.py`

Project Structure

In the project, the input image is passed in from the parent directory. You could try out with different images by placing them in the parent directory and then mentioning the input file path while calling the `hog_detector` binary file. The model files (the `weights.bin` and `bias.txt`) are both stored in the `project/models` subdirectory. These are the trained SVM files - you could use them to get the results. Also, the output file path could be mentioned while calling the `hog_detector` as well - the default value of it is in the parent directory. Along with the output image, a CSV file will be generated which contains the information about the inference happening in every frame – the detections, and the latency it took for detecting each frame.

6. Conclusions and Future Work

This project set out to answer a straightforward but genuinely interesting question: how much faster can a classical HOG+SVM rock-detection pipeline get if you throw parallelism at it? The short answer, it turns out, is a lot.

Starting from a sequential C++ baseline that averaged 1,160 ms per frame (under 1 FPS — essentially unusable for any real-time application), parallelizing the sliding-window loop with OpenMP brought the average down to 139 ms (roughly 7 FPS), an 8.4x speedup. Moving the entire computation to the GPU with CUDA pushed that further to 35 ms per frame, or about 29 FPS — a 33.5x speedup over the sequential baseline and 4x over OpenMP — all measured consistently over 300 frames. The p99 latency numbers tell an equally encouraging story: CUDA's worst-frame latency of 36 ms was tighter and more predictable than OpenMP's 180 ms, which suggests the GPU's throughput advantage also comes with better consistency. One of the features that worked out most was the CUDA kernel design: mapping a 3D grid of (window column x window row x 49 HOG blocks) meant every HOG block for every sliding-window position ran concurrently, and using eight warp-private shared-memory histograms per block instead of global atomics kept contention low. Correctness validation confirmed that the parallel feature vectors stayed within a 1e-3 absolute tolerance of the sequential reference,

meaning the parallelism doesn't compromise detection quality. That said, several things remain on the table.

One of the things that I wish I had completed in this project is video-based benchmarking, which could be a future scope — the project currently runs on static images, and testing at 720p and 1080p would let us empirically locate the crossover point where CUDA's PCIe transfer overhead is fully amortized by the sheer volume of sliding-window work. Hard negative mining was partially explored during SVM training, but a more rigorous round of it on diverse lighting conditions would improve the model's robustness before any real deployment. Longer term, comparing this classical pipeline against a lightweight deep learning detector at the same image resolution would make for a compelling study of where HOG+SVM still holds its own on resource-constrained hardware.

From a course perspective, this project touched some pillars of ME 759: the OpenMP phase applied collapse(2) loop parallelism and thread-private data structures to eliminate false sharing; the CUDA phase demanded careful thought about grid and block dimensionality, shared memory layout, atomic contention, and parallel tree reduction for the SVM dot product — concepts that were abstract in lecture but very concrete when a 30x speedup depends on getting them right.

References

- [1] <https://pyimagesearch.com/2014/11/10/histogram-oriented-gradients-object-detection/>
- [2] Wasala, Mateusz, and Tomasz Kryjak. "Real-time HOG+ SVM based object detection using SoC FPGA for a UHD video stream." In 2022 11th Mediterranean Conference on Embedded Computing (MECO), pp. 1-6. IEEE, 2022.